A I E N G I N E E R

Technical Challenge

Multi-Agent AI System for an Immersive Digital Showroom

Overview

You are being asked to design and build a small but production-minded multi-agent AI system from scratch. The scenario is grounded in a real product domain: a company uses a 3D virtual showroom to engage potential buyers of high-value products. An AI assistant lives inside that showroom and handles inbound conversations.

There is no existing codebase to extend. We have a defined stack — the tools we use in production — and we expect you to build with them. Where you have latitude is in how you decompose the agents, design the data strategy, and structure the system. That is where we evaluate your thinking.

Business Context

The product

A B2B SaaS platform for high-value product sales. Clients visit a 3D virtual showroom to explore product lines and space configurations before committing to a purchase.

The problem

Buyers ask a wide range of questions during a showroom visit. Some want product details. Some want to know if their available space suits a given configuration. Some are ready to buy. Some want a human. The assistant must route each intelligently and respond helpfully.

> **Key constraint**
> The assistant must be embeddable in a real-time 3D web interface and support voice interaction via the OpenAI Realtime API. Responses must be fast, structured, and channel-aware — the calling system decides how to render the response based on a channel field in the output.

What You Are Building

Build a multi-agent conversational AI system that:

- Classifies incoming user messages into distinct intent categories
- Routes each intent to a specialized agent capable of answering it

- Uses RAG (retrieval-augmented generation) to ground responses in provided knowledge bases
- Returns a structured JSON response on every turn
- Handles conversation state across multiple turns

Required Intent Categories

Your classification layer must handle at minimum the following intents. You may add more if you see fit — justify your choices in the architecture doc.

Intent	Trigger example	Expected channel
Product Info	*"What is the 0-60 time and range on the Aether GT Coupe?"*	`text`
Space Analysis	*"My showroom floor is 400 sqm. What layout do you recommend?"*	`text`
Purchase Intent	*"I want to reserve the Aether GT in matte black. How do I proceed?"*	`text`
Voice Request	*"Can you read that back to me?"*	`voice`
Escalation	*"I'd rather speak to a real person."*	`escalation`

Response Contract

Every agent response — regardless of intent — must conform to the following JSON structure:

```
{
  "message_agent": "string  — the text shown or spoken to the user",
  "channel":       "text | voice | escalation",
  "intent":        "string  — the classified intent for this turn",
  "session_id":    "string  — passed through from the request"
}
```

> **Why this matters**
> The 3D frontend renders responses differently depending on channel. text goes to the chat overlay. voice triggers a Realtime API session for bidirectional audio. escalation triggers a human handoff notification. A malformed or missing channel will silently break the UI.

Provided Data

You are given three plain-text knowledge base files to use as your RAG corpus. They are included in this package as .txt files. You must decide how to chunk, embed, and store them — that decision is part of the evaluation.

Source	Content type	Example query it should answer
vehicle_catalog.txt	Vehicle specs, features, trims, pricing tiers	"What is the horsepower and range of the Aether GT Coupe?"
dealership_faq.txt	Financing, test drives, warranty, delivery FAQs	"Do you offer financing or lease options?"
showroom_layouts.txt	Showroom floor layouts by sq footage and tier	"What layout fits a 400 sqm showroom floor?"

The files contain enough entries to demonstrate meaningful retrieval across all five intents. They are illustrative, not exhaustive. You may add synthetic entries to improve coverage, but clearly label anything you add.

Technical Requirements

Defined stack — use these

These are the tools we use in production. Build with them.

- Python 3.11+, async throughout
- OpenAI Agents SDK (openai-agents) as the orchestration layer
- OpenAI Vector Store + FileSearchTool for all RAG retrieval
- OpenAI Realtime API (wss://api.openai.com/v1/realtime) for the voice channel — you must have your own OpenAI account with Realtime API access

Hard functional requirements

These outcomes are non-negotiable regardless of stack.

- Multi-agent architecture — distinct agents with clear, separated responsibilities
- RAG-grounded responses — at least two intents must retrieve from the provided knowledge base files
- Structured JSON output on every response conforming to the contract above
- Multi-turn conversation state — the assistant must remember context within a session
- Realtime session must handle at minimum: session setup, user audio input, agent audio response, and graceful session teardown
- A working README — the evaluator must be able to run the project locally without asking questions

> **On deviating from the defined stack**
>
> If you believe a different tool or approach is technically superior for a specific component, you may deviate — but only if you include a written section in your architecture document titled 'Stack Deviations' that explains: (1) what you changed, (2) why the alternative is better for this use case, and (3) what the tradeoffs are. Deviations without justification will be scored as if the requirement was not met.

Good to have

- Clean separation between the agent orchestration layer and the voice/realtime session lifecycle
- Graceful fallback when the Realtime API session fails or times out — voice should degrade to text, not crash
- Async-first design throughout — justify any synchronous choices
- Error handling across all agents — one failing agent should not take down the whole system

> **What we do NOT require**
> A deployed production environment. A full frontend UI. Perfect RAG recall. AWS or any cloud infrastructure. We are evaluating your thinking and your code — run it locally, demo it locally.

Architecture Document

Include a written architecture document (PDF or Markdown) covering:

1. Agent decomposition — what agents exist, what each is responsible for, and why you drew the boundaries where you did
2. Workflow diagram — how a message flows from user input to final JSON response, including branching logic
3. RAG strategy — how you chunked and structured the knowledge base files and why
4. State management — how conversation history is maintained across turns
5. What you would change with more time — honest reflection on tradeoffs made

Deliverables

Deliverable	Format	Required
Multi-agent system	GitHub repository	Yes
Architecture document	PDF or Markdown in repo	Yes
Demo walkthrough	Recorded video, max 5 minutes	Yes

> **Submission**
> Send a link to your GitHub repository and video walkthrough to the contact who sent you this brief. No zip files. The repo must be publicly accessible or shared with the provided GitHub handle.

Good luck.